Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут комп’ютерних наук та інформаційних технологій
Кафедра автоматизованих систем управління



Звіт

до лабораторної роботи № 5

з дисципліни

Програмування мобільних смарт-систем

на тему:

«Локальне зберігання даних у БД Room та візуалізація динамічних процесів»

Виконав: студент групи ОІ-43
Гончаров Вадим
Прийняв: асистент каф. АСУ
Зварич Р.О.

Львів-2026

Мета роботи

Опанувати роботу з локальними базами даних в Android за допомогою бібліотеки Room (абстракція над SQLite) та навчитися візуалізувати часові ряди даних за допомогою графічних бібліотек.

Завдання на лабораторну роботу

1. Створити Entity SensorRecord з полями id, timestamp, sensorValue та actionTriggered.
2. Реалізувати DAO для асинхронного запису й отримання останніх N записів у хронологічному порядку.
3. Реалізувати singleton Room Database та Repository, не виконуючи операції БД у головному UI-потоці.
4. Записувати оброблену інтенсивність руху за таймером приблизно раз на секунду та пов’язувати записи з новими успішними активаціями актуаторів.
5. Створити окремий екран Analytics із лінійним графіком, де вісь X показує час, а вісь Y - інтенсивність руху в м/с².
6. Забезпечити реактивне оновлення графіка через Flow і StateFlow та обмежити відображення 50 найновішими точками за останню годину.

Технічний стек

Таблиця 1. Використані компоненти

Компонент	Версія або роль
Kotlin / Coroutines / Flow	2.3.21 / 1.10.2 / асинхронний та реактивний конвеєр
Room	2.8.5: Entity, DAO, Database, KTX
KSP	2.3.12: генерація Room implementation і JSON schema
Jetpack Compose / Material 3	Екран Dashboard, Analytics і навігація
Vico	3.3.1: Compose LineChart
JUnit / AndroidX Test	129 unit tests; Room DAO instrumented tests підготовлено

Хід виконання роботи

Етап 1. Схема Room

База sleep_tracker.db має версію 1. Єдина таблиця sensor_records містить обов'язкові поля методички. Поле timestamp зберігає Unix time у мілісекундах, а не монотонний SensorEvent.timestamp. Для часових запитів Room створює індекс index_sensor_records_timestamp.

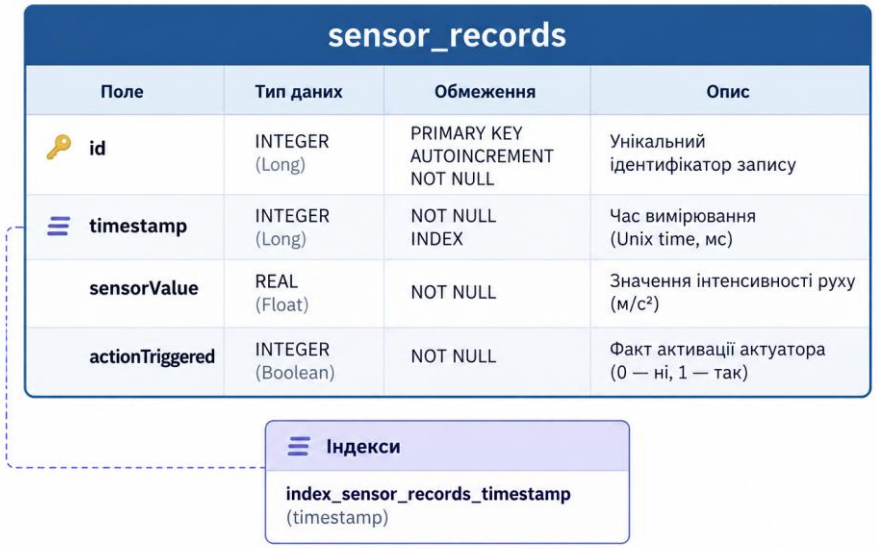


Рис. 1. ER-діаграма таблиці sensor_records

Лістинг 1. Entity SensorRecord

```
@Entity(tableName = "sensor_records", indices = [Index(value = ["timestamp"])])
data class SensorRecord(
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val timestamp: Long,
    val sensorValue: Float,
    val actionTriggered: Boolean,
)
```

Етап 2. DAO та порядок часових даних

DAO спочатку вибирає найновіші рядки через ORDER BY timestamp DESC, id DESC і LIMIT. Зовнішній SELECT повертає відібрані рядки за зростанням часу, тому графік не розвертає часовий ряд. Repository обмежує limit діапазоном 1..500 і перевіряє timestamp, NaN, Infinity та від'ємні значення.

Лістинг 2. Запит останніх N вимірювань

```
SELECT * FROM (
    SELECT * FROM sensor_records
```

```
    WHERE timestamp >= :sinceInclusive
    ORDER BY timestamp DESC, id DESC
    LIMIT :limit
) ORDER BY timestamp ASC, id ASC
```

Таблиця 2. Операції SensorRecordDao

Операція	Призначення
insert	Асинхронне додавання SensorRecord.
observeRecent	Flow останніх N рядків у хронологічному порядку.
recordsBetween	Вибірка заданого часового інтервалу.
observeCount	Реактивна кількість вимірювань.
observeActionCount	Реактивна кількість actionTriggered = true.
deleteOlderThan	Безпечна retention-операція за timestamp.

Етап 3. MVVM та Repository



Рис. 2. MVVM-конвеєр локальної історії

Room не викликається з Composable. SensorViewModel залишається єдиним джерелом MovementAnalysis. LocalHistoryRecorderViewModel спостерігає його StateFlow, тому перехід між Dashboard і Analytics не створює другого SensorManager listener та не зупиняє запис. Singleton Database отримує applicationContext і не використовує allowMainThreadQueries або destructive migration.

Етап 4. Отримання та збереження даних акселерометра

AndroidAccelerometerDataSource реєструє SensorEventListener із SENSOR_DELAY_UI та передає X, Y, Z і монотонний event.timestamp через

callbackFlow. SensorViewModel обробляє потік одним MovementClassifier. Класифікатор обчислює евклідову відстань між послідовними векторами, згладжує її коефіцієнтом 0,25, переходить у MOVEMENT при 0,45 м/с² і повертається у STILL при 0,20 м/с². У Room записується саме згладжена motionIntensity, а не модуль гравітації. LocalHistoryRecorderViewModel має один цикл у viewModelScope: раз на 1000 мс він бере останній валідний Active state, пропускає незмінений timestampNanos і створює Unix timestamp через System.currentTimeMillis(). RoomSensorHistoryRepository виконує insert у Dispatchers.IO.

Лістинг 3. Запис нового вимірювання

```
if (lastRecordedSensorTimestampNanos == sensorTimestamp) return
writeMutex.withLock {
    val actionForRecord = pendingActivation
    repository.addMeasurement(clock(), value, actionForRecord)
    lastRecordedSensorTimestampNanos = sensorTimestamp
    if (actionForRecord) pendingActivation = false
}
```

Етап 5. Зв’язок Room із подіями актуаторів

ActuatorViewModel надсилає ActuatorActivation через необмежений Channel лише після ActuatorOperationStatus.SUCCESS. Подіями є VIBRATION_PULSE, VIBRATION_ENABLED і FLASHLIGHT_ENABLED. Вимкнення, UNAVAILABLE, ERROR та повторне застосування вже активного persistent state не вважаються новою активацією. Recorder тримає pendingActivation до успішного insert, тому помилка Room не втрачає подію.

Етап 6. Реалізація екрана Analytics

AnalyticsViewModel кожну хвилину оновлює межу rolling one-hour window і через flatMapLatest підписується на три Room Flow. Графік отримує не більше 50 найновіших точок. X використовує фактичні нерівномірні timestamps як відносні секунди від першої точки, що усуває втрату точності Double; formatter відновлює локальний час. Y показує smoothed movement intensity у m/s².

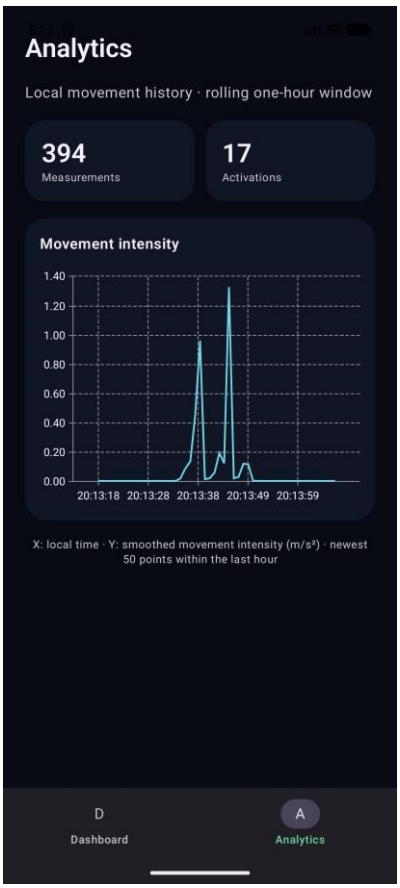


Рис. 3. Екран Analytics з реальними записами Room

На фактичному скриншоті відображено 394 вимірювання і 17 успішних активацій після instrumented test run та повторного запуску. Видимі піки створено командами virtual accelerometer. Дані залишалися доступними при BLE disconnected; Firebase не бере участі у локальному записі.

Етап 7. Тестування та результати

Таблиця 3. Фактично виконані перевірки

Перевірка	Результат
testDebugUnitTest	129 passed, 0 failed, 0 errors, 0 skipped (аудит 13.10.2026)
connectedDebugAndroidTest	Поточний повторний запуск не відбувся: No connected devices; раніше зафіксовано 3/3
assembleDebug	BUILD SUCCESSFUL (аудит 13.10.2026)
lintDebug	BUILD SUCCESSFUL; 0 errors (аудит 13.10.2026)

Під час першого runtime запуску Vico завершувався з IllegalArgumentException через надто точні абсолютні X-values. Перехід на

відносні секунди усунув crash. Окремо додано Material 3 Vico theme provider і background Analytics для читабельного dark-theme контрасту.

Статус доказів

Підтверджено фінальний runtime screenshot Analytics, згенеровану Room schema version 1, 129 успішних unit-тестів, debug-збірку та lint без помилок. 13.10.2026 застосунок запущено на емуляторі Pixel 10 Pro у debug-режимі; у Database Inspector відкрито sleep_tracker.db і таблицю sensor_records з фактичними записами та полями id, timestamp, sensorValue, actionTriggered. Перевірку на фізичному пристрої не виконано.

Класифікація результатів перевірки

Категорія	Фактичний результат
VERIFIED BY UNIT TESTS	129 tests, 129 passed, 0 failed, 0 errors, 0 skipped. LocalHistoryRecorderViewModelTest перевіряє throttling, одноразове actionTriggered, злиття кількох активацій у Boolean та retry після помилки insert.
VERIFIED BY BUILD/LINT	13.10.2026: testDebugUnitTest, assembleDebug і lintDebug завершилися BUILD SUCCESSFUL; lint має 0 errors.
VERIFIED ON EMULATOR	Наявний реальний скриншот Analytics із хвилями, 394 вимірюваннями та 17 записами з активацією. Під час реалізації перевірено створення sleep_tracker.db і збереження після force-stop/start без clear data.
VERIFIED ON PHYSICAL DEVICE	Немає підтвердження.
NOT VERIFIED	Поточний connectedDebugAndroidTest не повторено; перевірку на фізичному пристрої не виконано. Database Inspector підтверджено окремо на емуляторі.

Відповідність методичним вказівкам

Вимога	Статус	Доказ
SensorRecord: id, timestamp, sensorValue, actionTriggered	Виконано	Entity sensor_records; timestamp index
DAO: запис і останні N записів	Виконано	suspend insert; Flow; DESC selection та ASC output
Singleton Room Database	Виконано	sleep_tracker.db, version 1, exportSchema = true
MVVM та Repository	Виконано	SensorViewModel -> recorder -> repository -> DAO
Динамічний LineChart	Виконано	Vico, Room Flow, StateFlow, максимум 50 точок
Скриншот графіка з коливаннями	Виконано	Рис. 3, реальний emulator screenshot
Database Inspector	Виконано	Рис. 4, реальний кадр App Inspection: sleep_tracker.db -> sensor_records

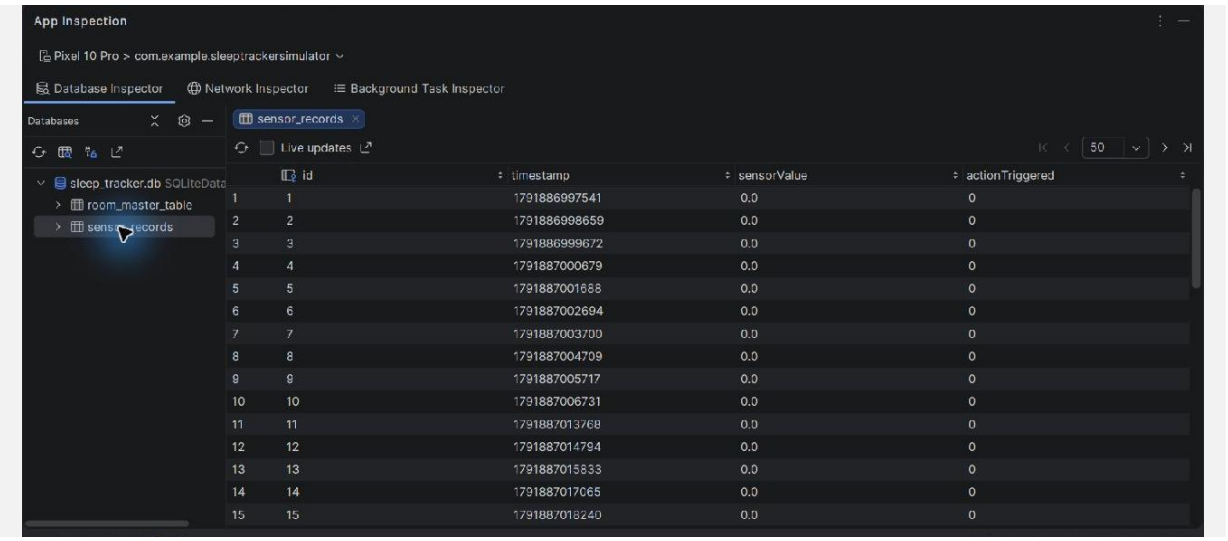


Рис. 4. Таблиця sensor_records у Android Studio Database Inspector

Висновки

Room практичніший за пряме керування SQLite, оскільки перевіряє SQL під час компіляції, генерує типобезпечний DAO, інтегрується з suspend і Flow та централізує версію схеми. Це зменшує кількість ручного Cursor-коду і помилок перетворення типів.

Локальна база підвищує автономність Sleep Tracker Simulator: рухова історія записується при відсутньому BLE та незалежно від Firebase або

Інтернету, а SQLite зберігає її після завершення процесу. Room Flow автоматично перевидає дані після insert, тому AnalyticsViewModel і Vico оновлюють графік без polling та без запитів із Composable.

Історію актуаторів реалізовано як одноразові типізовані події успішної активації. Mutex пов’язує pending event із наступним успішним SensorRecord, а помилка insert не очищає pending flag. Під час аудиту 13.10.2026 виконано 129 unit-тестів без помилок, assembleDebug і lintDebug. Окремо підтверджено роботу debug-застосунку на емуляторі та наявність записів у sensor_records через Android Studio Database Inspector. Перевірку на фізичному пристрої не виконано.